

C 语言程序设计

第六讲 数组 一维数组的定义与应用



张 华

一维数组的定义与应用

■ 数组的概念

■ 一维数组

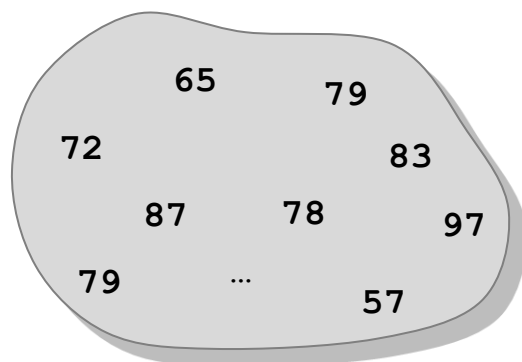
✿ 概念，定义，初始化，典型应用

■ 程序设计举例

简介

引例：学生成绩管理程序

问题：保存n个学生的成绩，并按从高到低的顺序排序。



存储

问题：数据分散，无法批量操作

score1
score2
...
scoren

要多少个变量
怎么进行排序
怎么进行查找

?

存储



score

score[0] score[1]

...

score[n-1]

score[i] i=0,1,...,n-1

优点：数据集中管理，方便批量操作。

用一个名字和编号
去引用这些变量。

数据结构

- **数据结构 (Data Structure) 是计算机存储、组织数据的方式。**
 - ✿ 它是指相互之间存在一种或多种特定关系的数据元素的集合。
 - ✿ 简单来说，数据结构定义了数据如何被存储、访问和操作，以便高效地解决各种问题。
- **不同的数据结构适用于不同的场景，选择合适的数据结构可以大大提高程序的运行效率和代码的可读性。**
- **数据结构是算法设计的基础，不同的数据结构会影响算法的实现方式、运行效率与存储空间需求。**

数组是一种常用的数据结构

数组

- 一种线性数据结构，由一组相同类型的元素组成，在内存中占据连续的存储空间。
- 通过索引，能快速访问数组中的元素，时间复杂度为 $O(1)$ 。
 - 例如，在处理大量同类型的数值数据时，数组就非常适用。
- 但数组的大小通常在初始化时就已确定，后续若要增加或删除元素，可能需要重新分配内存，操作相对复杂。

91	80	76	78	64	88	75	69	...	77	scores
scores[0]	scores[1]							...	scores[n-1]	

C语言的数组

数组

✿ 由一系列类型相同的变量（元素）构成。

- 数组元素都属于同一种数据类型。
- 数组元素的数目是固定且有限的。
- 在内存中数组元素是连续存放的。

✿ 引用数组元素的格式：

数组名[位置编号或下标]

- 下标运算符：**[]**
 - 优先级为**1**，左结合。
- 第一个位置编号是 **0**。
- 数组 **c** 的 **12** 个元素如图所示。

	c	数组名
c[0]	-45	数组元素
c[1]	6	
c[2]	0	
c[3]	72	
c[4]	1543	
c[5]	-89	
c[6]	0	
c[7]	62	
c[8]	-3	
c[9]	1	
c[10]	6453	
c[11]	78	

数组元素引用名

C语言的数组

数组元素

✿ 数组元素可视为普通变量。

```
c[0] = -45;  
scanf("%d", &c[1]);  
printf("%d", c[1]);
```

✿ 下标可以是整型常量或整型表达式。

```
若 x = 3, 那么  
c[5-2] == c[3] == c[x]
```

	c
c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

定义数组

定义数组的格式

类型说明符 数组名 [整型常量表达式]

数组类型 数组名 元素个数（数组长度）

```
#define MAX 200

int c[12];
float f[1000];
int a[MAX], b[MAX*10];
```

数组长度必须是一个整型常量或整型常量表达式

```
int n, m=5, x[m];
scanf("%d", &n);
{
    int y[n];
}
```

数组长度不能是不确定的值

数组的初始化

数组的初始化

类型说明符 数组名 [整型常量表达式] = {初值表};

✿ 在定义数组时为每一个元素指定初值。

✿ 举例

```
int a[5] = {1, 2, 3, 4, 5, 6};
```

初值表中的初值太多，产生语法错误（syntax error）

```
int a[5] = { };
```

Expression expected

初值表不能为空

```
int a[5] = {1};
```

a[0]=1，其他元素的初值是 0

数组的初始化

数组的初始化说明

- ✿ 若不指定数组的长度，就是初值表中初值的个数

```
int a[ ] = {1, 2, 3, 4, 5};
```

- ✿ 程序块内没有初始化的数组，其元素的值不确定

```
int a[5]={1};  
for (i=0;i<5;i++)  
    printf("\ %d",a[i]);
```

1 0 0 0 0

```
int a[5];  
for (i=0;i<5;i++)  
    printf("\ %d",a[i]);
```

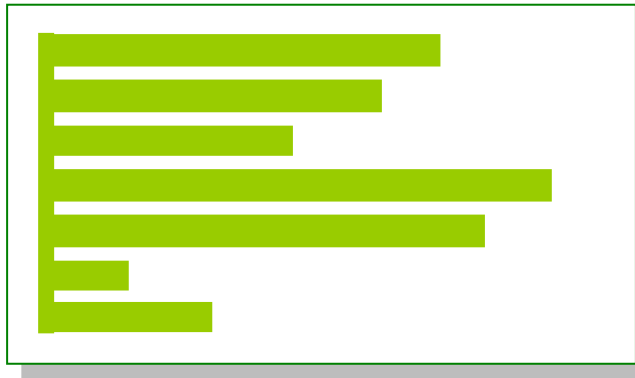
872 0 1492 4160 186

不可预知

案例分析一：使用数组

问题

✿ 从一个数组中读取数据，并以直方图的形式显示信息。



用一系列 ‘*’ 来代替长方形

```
*****  
***  
*****
```

源代码 (cw1001.c)

```
#include <stdio.h>  
#define SIZE 10  
  
void main() {  
    int n[SIZE] = {19, 2, 15, 7, 11, 9, 13, 5, 17, 1};  
    int i ,j;
```

案例分析一：使用数组

源代码

```
printf("%s%13s%17s\n", "Element", "Value", "Histogram");

for (i=0; i<=SIZE-1; i++) {

    printf("%7d%13d          ", i, n[i]);

    for (j=1; j<=n[i]; j++)
        printf("%c", '*');

    printf("\n");
}
```

Element	Value	Histogram
0	19	*****
1	3	***
2	15	*****
3	7	*****
4	11	*****
5	9	*****
6	13	*****
7	5	*****
8	17	*****
9	1	*

案例分析二：使用数组

问题

统计全班32名同学某门功课的平均成绩，找出最高分。

➤ 把学生成绩保存到数组 **score**

源代码 (cw1002a.c)

```
#include <stdio.h>
#define MAX 32

void main()
{ float score[MAX], sum, best;
  int i;

  printf("Input %d scores:\n", MAX);

  for (i=0; i<MAX; i++)
    scanf("%f", &score[i]);
```

保存最好成绩

数据结构

输入数据

案例分析二：使用数组

源代码

遍历数组进行

- 累加
- 比较

输出结果

```
sum=best=score[0];

for (i=1;i<MAX;i++)
{
    sum+=score[i];

    if (best<score[i])
        best=score[i];
}

printf("The average: %.1f\n", sum/MAX);
printf("The best: %.1f\n", best);
}
```

案例分析三：使用数组

问题

统计全班32名同学某门功课的平均成绩，找出最高分。

➤ 使用位置指针变量

源代码 (cw1002b.c)

```
#include <stdio.h>
#define MAX 32

void main()
{ float score[MAX], sum;
  int i, best_p;

  printf("Input %d scores:\n", MAX);

  for (i=0; i<MAX; i++)
    scanf("%f", &score[i]);
```

65	72	83	79	97	87	79	57	91	78
----	----	----	----	----	----	----	----	----	----



位置指针

保存最好成绩在
数组中的位置

案例分析三：使用数组

源代码

```
sum=score[0];
best_p=0;

for (i=1; i<MAX; i++)
{
    sum+=score[i];

    if (score[best_p]<score[i])
        best_p=i;
}

printf("The average: %.1f\n", sum/MAX);
printf("The best: %.1f\n", score[best_p]);
}
```

适用于数组元素较宽（复杂）的情况

案例分析四：使用数组

问题

✿ 把一个任意的正整数 N 转换成 d 进制数。

用 N 除以 d
如果商不为 0，继续用商除以 d ，直到商为 0

➤ 用数组 `remainder[MAX]` 保存每一步的余数

N	N/8	N%8
1348	168	4
168	21	0
21	2	5
2	0	2

Diagram illustrating the conversion process:

- A light blue arrow points from the `N%8` column to the label `remainder[len]`.
- A vertical dashed line separates the table from the array elements.
- Four horizontal arrows point from the `N%8` column to the array elements: `remainder[0]`, `remainder[1]`, `remainder[2]`, and `remainder[3]`.
- A vertical dashed line with an upward arrow is to the right of the array elements, labeled `print`.

案例分析四：使用数组

源代码 (cw1003.c)

```
#include <stdio.h>
#define MAX 10 /* 估计的最大位数 */

void main() {
    int N, /* 保存正整数和每次相除得到的商 */
        d, /* 基数 */
        len; /* 保存实际的位数 */
    int i;
    int remainder[MAX];

    printf("Input a decimal integer and a base:");
    scanf("%d%d", &N, &d);

    printf("Converting...\n");
```

案例分析四：使用数组

源代码

```
len=0;
while (N) // 只要商 (N) 不为零
{
    remainder[len]=N%d;
    len++;
    N=N/d;
}

printf("equals ");

for (i=len-1; i>=0; i--) printf("%d", remainder[i]);
}
```

```
Input a decimal integer and a base:1348 8
Converting...
equals 2504
```

案例分析五：使用并列数组

约瑟夫问题

- ✿ m 个人围成一圈，从第一个人开始报数，数到 n 的人出圈。再由下一个人报数，数到 n 的人出圈，……输出依次出圈的人的编号。

分析与设计

数据结构

数组 a 保存 m 个人的编号



数组 b 保存 m 个人的状态



0表示未出圈，1表示已出圈

并列数组

算法

- 顺序循环扫描，同时判断各人的状态
 - 未出圈，则报数
 - 数到 n 则出圈，即更新状态

案例分析五：使用并列数组

源代码 (cw1004.c)

```
#include <stdio.h>

#define M 5
#define N 3

void main() {
    int a[M];
    int b[M]={0}; //初始化状态：未出圈（0）
    int i, /* 顺序循环扫描的序号，同时用来计算下标 */
        count, /* 报数：0-1-2.....N-1 */
        out; /* 出圈人数 */

    //初始化编号
    for (i=0; i<M; i++)
        a[i]=i+1;
```

案例分析五：使用并列数组

源代码 (cw1004.c)

```
//顺序扫描，i保存扫描序号
for (i=0,count=0,out=0; out<M; i++)
{
    //判断当前扫描对象是否未出圈
    if (b[i%M]==0)
    {
        //如果未出圈，则判断报的数是否是N-1
        if (count%M==N-1)
        {
            //如果报到了N-1，则出圈，输出出圈顺序号和原始编号
            printf("%2d: %2d\n", out+1, a[i%M]);
            b[i%M]=1; //修改状态为“已出圈”
            out++; //出圈人数增一
        }
        count++; //计算下一个该报的数
    }
}
```

案例分析五：使用并列数组

运行结果

```
序号: 原编号  
1:    3  
2:    1  
3:    5  
4:    2  
5:    4
```

小结

 数组是由同一种数据类型的元素序列构成的。

- ✿ 数组元素按顺序存储在内存中。
- ✿ 通过使用整数类型的下标来访问数组元素。
- ✿ 程序员要能正确地使用数组下标，因为编译器和程序运行时都不检查索引是否合法。
- ✿ C不支持把整个数组作为函数的参数进行传递，但是可以传递数组的地址。然后函数可以利用该地址来处理原始数组。

练习

跳水比赛评分

- ✿ 假设在跳水比赛中，有**8**个评委对参赛选手每一次跳水打分，参赛选手一次跳水的最终得分是去掉一个最高分和一个最低分后的平均分。
- ✿ 编写程序输入**8**位评委的评分，计算选手的得分。

练习

考试成绩分段统计

- ✿ 输入C语言课程期中考试成绩。
- ✿ 统计不及格人数，统计及格成绩每10分一段的人数。